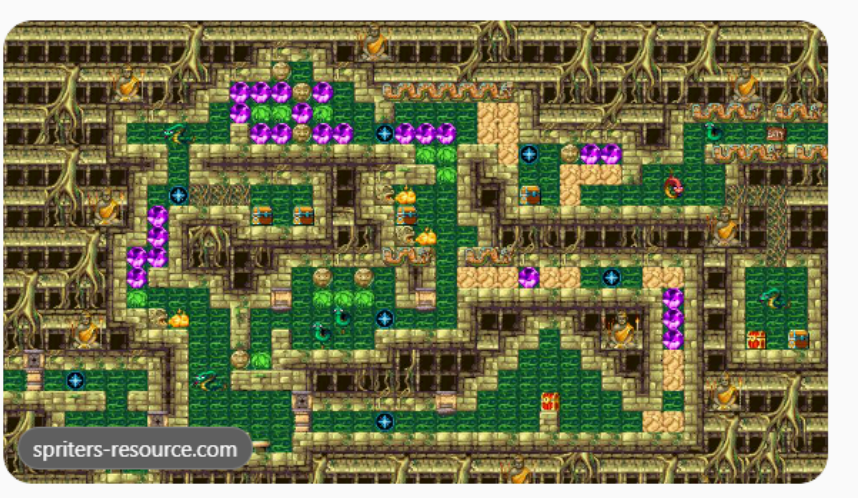
TP Final Integrador: “Rescate en la Ciudad Perdida”

1. Contexto del problema

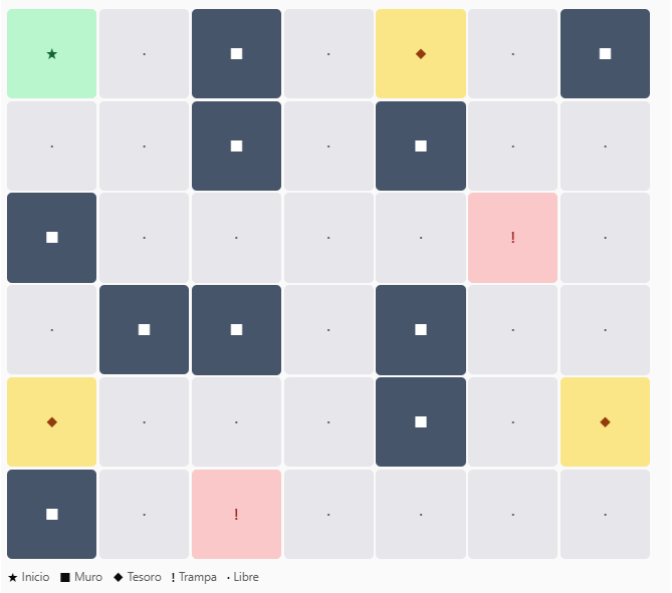
Un grupo de exploradores ha descubierto una ciudad antigua que contiene tesoros, obstáculos y zonas peligrosas. Para recuperar los tesoros, deberán recorrer diferentes sectores representados mediante una matriz.



Cada sector de la ciudad está formado por celdas que pueden contener caminos libres, muros, tesoros, trampas o la posición de un explorador.

El desafío consiste en desarrollar un sistema que permita representar la ciudad, administrar sus elementos, explorar sectores mediante algoritmos recursivos, registrar los resultados de las expediciones y comparar la eficiencia de diferentes estrategias de ordenamiento.

2. Representación de la ciudad



La ciudad se representa mediante una matriz de $N \times M$ sectores.

Cada sector puede tener diferentes atributos

Cada sector tiene una categoría que puede ser tesoro, trampa, o camino libre. Además, hay un sector especial que es el punto de inicio.
Además de sectores, existen muros, por los que no se puede transitar. Los muros están representados como valores nulos en la matriz.

Ejemplo de la visualización de una ciudad de 6 × 7:

3. Actividades a desarrollar

Parte A. Modelado mediante TDA

Diseñar e implementar los siguientes Tipos de Datos Abstractos:

TDA Sector

Los distintos sectores tienen atributos propios que definen el comportamiento al llegar a ese sector. Un muro no es transitable, por lo cual al llegar a un muro se deberá volver al punto anterior y buscar un sector transitable. Un tesoro tiene una recompensa que suma al puntaje de un explorador. Una trampa tiene un descuento en el puntaje de un explorador.

TDA Ciudad

Debe permitir administrar la matriz que representa la ciudad. Como mínimo, deberá ofrecer operaciones para crear una ciudad de las dimensiones solicitadas por el usuario, consultar el contenido de una celda, modificar una posición válida, contar tesoros y determinar si una celda es transitable.

TDA Explorador

Deberá almacenar un identificador, nombre, posición actual, cantidad de tesoros recolectados, cantidad de trampas encontradas y puntaje acumulado.

TDA Expedición

Permitirá registrar los resultados de una exploración: identificador, explorador participante, tesoros recuperados, celdas visitadas, puntaje y tiempo de ejecución.

Parte B. Procesamiento de matrices

Implementar las siguientes funcionalidades:

1. Mostrar el mapa de la ciudad.
2. Calcular la cantidad de tesoros y trampas por fila y por columna.
3. Identificar la fila y la columna que contienen mayor cantidad de tesoros.
4. Determinar cuántas celdas transitables existen.
5. Permitir modificar el contenido de una celda, verificando que la operación sea válida.
6. Generar un informe general del estado de la ciudad.

Parte C. Exploración recursiva

Implementar una función recursiva que, partiendo de la posición del explorador, determine cuántos tesoros son alcanzables.

Reglas de exploración:

- Solo se permite avanzar hacia arriba, abajo, izquierda o derecha.
- No se pueden atravesar muros.
- Las trampas son transitables, pero deben registrarse.
- Cada celda debe procesarse una sola vez.
- No se permite salir de los límites de la matriz.

La función debe devolver la cantidad de tesoros alcanzables y registrar cuántas celdas fueron visitadas.

Restricción: la exploración deberá resolverse mediante recursión, sin utilizar estructuras de datos avanzadas.

Se solicitará además explicar el caso base, el caso recursivo y cómo se evita visitar indefinidamente una misma posición.

Parte D. Ordenamiento de expediciones

Una vez realizadas varias expediciones, el sistema deberá permitir ordenar los resultados según distintos criterios:

- Mayor puntaje obtenido.
- Mayor cantidad de tesoros recuperados.
- Menor tiempo de ejecución.
- Menor cantidad de celdas visitadas.

Se deberán implementar dos algoritmos. Uno de cada tipo: Fuerza Bruta (selección, inserción o burbuja mejorado) y Divide y Venceras (Heap Sort, Quick Sort, Merge Sort).

Parte E. Medición y comparación de tiempos

Realizar experimentos para comparar el comportamiento de los dos algoritmos de ordenamiento implementados.

Se deberán generar conjuntos de expediciones de diferentes tamaños y considerar tres situaciones: datos inicialmente ordenados, ordenados en sentido inverso y desordenados.

Ejemplo de Tabla de resultados a completar

Cantidad de expediciones	Algoritmo A (ms)	Algoritmo B (ms)
100	—	—
500	—	—
1.000	—	—

2.000	—	—
5.000	—	—

Para cada tamaño y situación deberán realizarse al menos cinco mediciones y calcular el promedio.

A partir de los resultados, responder:

1. ¿Cuál de los algoritmos fue más rápido?
2. ¿El orden inicial de los datos influyó en los tiempos?
3. ¿Qué ocurrió al aumentar la cantidad de expediciones?
4. ¿Los resultados experimentales coinciden con el comportamiento esperado de cada algoritmo?
5. ¿Qué factores pueden afectar la precisión de las mediciones?

Se deberá presentar al menos un gráfico comparativo de tiempos.

4. Requisitos generales

El programa deberá contar con un menú que integre las funcionalidades anteriores, validar los datos ingresados, separar las operaciones de los TDA de la interacción con el usuario y evitar el uso de variables globales innecesarias.

Los algoritmos de ordenamiento deberán recibir conjuntos equivalentes de datos para que la comparación sea válida.

Se entregará un conjunto de datos iniciales con los que se deberán poblar los TDA al comenzar la ejecución del sistema de manera que no haya que cargarlos en forma manual para las pruebas iniciales.

Se entregará el esqueleto del programa principal y las llamadas correspondientes a módulos con nombres y parámetros que se deberán respetar. Podrán agregarse más métodos y atributos en los TDA, pero los existentes se deberán respetar y completar con lo solicitado en los comentarios.

5. Entrega

Se deberá presentar el código fuente, un informe breve que describa los TDA y los algoritmos utilizados, los casos de prueba realizados, la tabla y el gráfico de tiempos, y las conclusiones obtenidas.